

Function Calling 最初解决的是一个很直接的问题：模型不能只生成文本，还要能调用外部工具。它的典型形态是：模型判断需要哪个工具，生成结构化参数，系统执行函数，再把结果返回模型。但 Codex CLI 呈现出来的形态明显不一样。==它当然有 Function Calling 的特征，例如模型会读取文件、编辑代码、运行命令、获取执行结果；但它不是把这些能力包装成几个孤立函数，而是把它们组织成一个面向软件工程任务的产品系统。

所以，理解 Codex CLI，不能说它是“高级版 Function Calling”，而应说：**它是 Function Calling 在真实工作场景中产品化之后的形态变化。**

一、从“调用函数”到“进入工作现场”

普通 Function Calling 更像模型在远程调用一个 API。用户问问题，模型选择工具，系统返回结果。它的核心单位是“函数”。

但 Codex CLI 的核心单位不是函数，而是“工作现场”。这个现场包括代码仓库、文件系统、终端、依赖环境、测试命令、项目规范、Git diff 和用户确认。OpenAI 对 Codex CLI 的定位是运行在本地电脑上的 coding agent；这意味着它不是一个远程问答工具，而是进入开发者真实工作区的执行者。

这带来一个产品创新：**Function Calling 不再是“调用外部能力”，而是“让模型在一个受控环境中行动”。**

例如，传统 Function Calling 可能设计成：

```
read_file(path)
edit_file(path, content)
run_command(command)
```

但 Codex CLI 的用户感知不是“我让模型调用三个函数”，而是：

```
我给 Codex 一个任务；
它自己看项目；
找到相关代码；
修改文件；
运行测试；
根据错误继续修；
最后告诉我改了什么。。
```

这就是产品形态的变化。底层可能仍然离不开工具调用，但用户购买的不是工具调用，而是“任务推进能力”。

二、从“工具列表”到“能力边界”

Function Calling 的一个常见设计方式，是把很多工具暴露给模型，然后让模型自己选。但真实落地时，这种方式很危险：工具越多，越难控；权限越大，越容易出事故。

Codex CLI 的思路不是简单增加工具，而是先定义边界。比如 Codex 的 sandbox 文档把运行环境边界明确分为不同模式：只读、工作区可写、以及更高权限模式；sandbox 决定 Codex 可以自己做什么，approval policy 决定什么时候必须停下来问用户。

这说明 Codex CLI 的产品重点不是“模型能调多少函数”，而是“模型在什么边界内可以自主行动”。

这对 Function Calling 产品设计很有启发。普通 Function Calling 常常从工具设计开始问：

我要给模型哪些函数？

Codex CLI 展示的更成熟问题是：

我要给模型一个什么样的行动空间？
在这个空间内，哪些动作可以自动完成？
哪些动作必须确认？
哪些资源永远不能碰？？

这就是从“工具列表思维”升级为“能力边界思维”。

三、从“一次调用”到“任务循环”

Function Calling 的经典流程通常是一次或少数几次工具调用。但软件工程任务天然不是一次调用能完成的。写代码的真实过程是循环式的：

观察 → 修改 → 运行 → 报错 → 再观察 → 再修改 → 再验证

Codex CLI 的创新在于，它把 Function Calling 放进了这个循环里。读文件不是终点，读完要改；改文件不是终点，改完要跑测试；测试失败不是失败，而是下一轮输入。

这和普通 Function Calling 的产品逻辑有本质区别。普通 Function Calling 更像：

工具返回结果 → 模型回答用户

Codex CLI 更像：

工具返回结果 → 模型判断下一步动作 → 再调用工具 → 直到任务可验收

这里的关键产品创新是：**工具结果不只是回答素材，而是下一步行动的反馈信号。**

比如测试报错、lint 结果、类型检查失败、文件 diff，都不是给用户看的最终答案，而是给 Agent 继续工作的中间反馈。**Function Calling 在这里从“信息获取机制”变成了“控制回路的一部分”。**

四、从“模型判断”到“系统裁决”

普通 Function Calling 容易产生一个误区：模型决定调用什么工具，模型生成参数，系统照做。这在 Demo 阶段可行，在企业级产品中很危险。

Codex CLI 的产品设计体现出更成熟的原则：**模型可以提出动作，但系统负责裁决动作能不能执行。**

这就是 sandbox 和 approval 的意义。模型想执行某个命令，不代表它一定能执行；模型想修改某些文件，不代表它一定有权限；模型想访问网络，也可能需要用户确认。OpenAI 文档也强调，sandboxing 和 approvals 是配合工作的两个控制：sandbox 定义技术边界，approval policy 决定越界前是否要询问。

这对企业级 Function Calling 非常重要。模型不能成为权限主体，模型只能是意图生成器。真正的权限主体应该是用户、组织、项目和系统策略。

所以，从产品经理角度，Codex CLI 给出的启发是：

不要让模型“拥有工具”，而要让模型“申请动作”。

模型说：“我建议运行这个命令。”

系统判断：“这个命令是否在沙箱内？是否需要确认？是否有风险？”

用户或策略决定：“允许或拒绝。”

这比单纯设计 function schema 更接近生产系统。

五、从“prompt 约束”到“项目规则约束”

普通 Function Calling 里，很多人会把规则写进 prompt：不要乱改文件、先跑测试、遵守项目风格。但 prompt 是软约束，容易遗忘，也不利于团队复用。

Codex CLI 的一个重要产品思路，是把项目规则显式文件化，例如通过 AGENTS.md 让 Agent 在进入仓库时读取项目规范、测试命令、代码风格和禁止事项。OpenAI 文档说明，Codex 会使用 AGENTS.md 为仓库建立项目级指导；Agent Skills 则可以把 instructions、resources 和 optional scripts 打包成可复用能力，让 Codex 更可靠地遵循某类工作流。

这说明 Function Calling 的应用边界被扩展了。schema 约束的是“单次工具调用”，而项目规则约束的是“整个任务过程”。

这是一个非常重要的产品创新：**Agent 不再只依赖临时 prompt，而是被嵌入项目制度中。**

对企业产品来说，这意味着未来每个业务系统都可能需要自己的“Agent 操作手册”：

这个系统有哪些数据？
哪些字段是敏感字段？
哪些接口只读？
哪些动作必须审批？
失败时如何降级？
生成结果必须符合什么格式？

这比单纯设计几个 function call 更重要。

六、从“工具执行”到“可验收结果”

Function Calling 的输出常常是模型回答一句“已经完成”。但在真实工作中，“完成”不能由模型自己宣布。尤其是代码场景，完成要通过 diff、测试、构建、lint、类型检查、人工 review 来判断。

Codex CLI 的产品价值在于，它把“验收标准”纳入了 Agent 工作流。模型修改代码后，通常需要运行测试或检查结果；失败就继续修；最后用变更摘要和测试结果向用户交付。**这种设计让 Function Calling 不再停留在“工具调用成功”，而是走向“任务结果可验收”。**

这对产品经理非常关键。评估 Agent 产品不能只看：

调用工具是否成功？
模型回答是否自然？

而要看：

用户目标是否达成？
中间步骤是否合规？
结果是否可验证？
失败是否可恢复？

Codex CLI 的创新就是把“可验收性”前置到了产品逻辑里。

七、Codex CLI 对 Function Calling 应用的真正启发

从 Function Calling 的应用角度看，Codex CLI 提供的不是一个更复杂的 function call 示例，而是一个新的产品范式：

第一，Function Calling 的终局不是工具调用，而是任务执行。

用户不是为了让模型调用 `read_file` 或 `run_command`，而是为了让它修 bug、写功能、解释代码、完成迁移。

第二，工具不是越开放越好，而是要变成受控工作环境。

真正的产品能力，不是暴露万能工具，而是定义哪些动作可以自动做、哪些动作必须确认、哪些

动作永远不能做。

第三，工具结果不是答案素材，而是反馈信号。

在 Agent 产品中，工具返回值要进入下一轮决策，形成持续迭代，而不只是生成最终回复。

第四，Function Calling 必须和权限、上下文、规则、验收绑定。

单独的 function schema 只能解决参数格式问题，解决不了企业级落地里的安全、责任、成本和结果质量问题。

第五，产品经理设计的不是“函数”，而是“人机协作边界”。

Codex CLI 的本质不是替代开发者，而是重新划分人和 Agent 的分工：Agent 负责执行可验证的局部工程任务，人负责目标设定、边界确认、架构判断和最终验收。

结论

所以，Codex CLI 之所以看起来有很明显的 Function Calling 特征，但落地时又不太一样，是因为它已经越过了 Function Calling 的原始形态。

原始 Function Calling 是：

模型 → 函数 → 结果

Codex CLI 的产品逻辑是：

用户目标 → 项目上下文 → 受控行动空间 → 多轮工具反馈 → 权限与审批 → 测试和 diff 验收 → 人类最终确认

它真正创新的地方，不是“用了 Function Calling”，而是把 Function Calling 放进了一个完整的工程工作系统中。它让我们看到，企业级 Agent 产品的关键不在于给模型多少工具，而在于如何设计一个让模型可以安全行动、持续修正、结果可验收的人机协作环境。